

IPre Sueño-Vigilia y Compositional Learning

(IMT2985)

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl

Contenidos

I	Semana 0	4
1.1.	Introducción y Definiciones Básicas	4
	Lenguaje Formal	4
	Entropía de Shannon (Teoría de la Información)	4
	Principio de Longitud de Descripción Mínima (MDL)	4
	Encoders y Autoencoders	4
1.2.	Reinforcement Learning	4
	Una Introducción al Reinforcement Learning	4
	Elementos del Reinforcement Learning	4
	Procesos de Decisión de Markov Finitos	5
1.3.	DreamerV3	8
1.4.	Notas de Composicionalidad	8
	¿Qué no entendí?	8
1.5.	Referencias Semana 0	9
II	Semana 1	10
2.1.	Complejidad de Kolmogorov	10
	Teoría de la Información	15
2.2.	MDL	15
2.3.	Cálculo Lambda	15
2.4.	DreamCoder	15
	Domain-Specific Lenguajes (DSL)	15
	Síntesis Inductiva de Programas	15
	Typed Grammars	15
	Connection to Helmholtz Machine and VAEs	15
	Resultados Experimentales	15
2.5.	Flat Minima	15
	Resultados Experimentales	15
2.6.	Glosario de la Semana	15
	Weight Decay	15
	Hinton & van Camp	15
	Optimal Brain Surgeon / Damage (OBS / OBD)	15
	Algoritmo de Gibbs	15
	Distancia de Kullback-Leibler	15
III	Semana 2	16
3.1.	Searching Latent Program Spaces	16
	Background	16

Arquitectura LPN	16
Experimentos y Resultados	18
Limitaciones	19
Preguntas	19
Glosario	19
3.2. Gradient-Based Program Synthesis with Neurally Interpreted Languages	19
Formalización	19
Inferencia Discreta Secuencial	20
Preguntas	20
Glosario	21
3.3. Diferencias entre ambas investigaciones	21

I Semana 0

1.1. Introducción y Definiciones Básicas

Lenguaje Formal

Entropía de Shannon (Teoría de la Información)

Principio de Longitud de Descripción Mínima (MDL)

Encoders y Autoencoders

VAEs

1.2. Reinforcement Learning

Esta sección está basada en los capítulos 1 y 3 del libro *Reinforcement Learning: An Introduction* de Richard S. Sutton y Andrew G. Barto (2014-2015).

Una Introducción al Reinforcement Learning

El **Reinforcement Learning** (RL) es un acercamiento *computacional* al *aprendizaje por interacción*, el cual pareciera subyacer a cualquier teoría del aprendizaje e inteligencia. La guía de este tipo de aprendizaje son las metas o retornos que, por lo general, son a largo plazo.

Por un lado, es diferente al **Aprendizaje Supervisado** (*Supervised Learning*), el cual consiste en aprender, a partir de un conjunto de entrenamiento etiquetado, la manera de clasificar o regresionar los nuevos ejemplos, sin embargo, no se basa en la interacción con el ambiente, como sí lo hace el RL.

Por otro lado, el RL es diferente al **Aprendizaje No Supervisado** (*Unsupervised Learning*), el cual consiste —en esencia— en encontrar patrones o *estructura* en un conjunto de entrenamiento sin etiquetas, no obstante, el RL no busca una estructura, sino que maximizar el retorno.

De esta manera, este modelo de aprendizaje lo asumiremos como un tercer paradigma del **Aprendizaje de Máquina** (*Machine Learning*).

Un reto que aparece en el RL es el tradeoff entre *explorar* y *explotar*: explorar para realizar mejores acciones a futuro; explotar decisiones que se sabe que son provechosas.

Finalmente, el RL es parte de una tendencia en la IA de retorno a principios generales y simples. Los métodos basados en principios generales se conocen como **métodos débiles** («*weak methods*»), mientras que aquellos basados en conocimiento específico se denominan **métodos fuertes** («*strong methods*»).

Elementos del Reinforcement Learning

El ciclo de aprendizaje en el RL tiene dos actores principales: el **agente** y el **ambiente**. En esencia, el agente toma decisiones buscando una manera de actuar dado cierto estado y el ambiente, el objeto con el que el agente interactúa, se encarga de indicarle una recompensa dado su actuar y el siguiente estado en que estará.

De manera más técnica, un sistema de RL se compone de: una *política*, una *señal de recompensa*, una *función de valor* y, opcionalmente, un *modelo del ambiente*.

Una **política** (*policy*) es un mapeo (que denota una probabilidad) que el agente aprenderá y que define la manera en que el learner se comporta en determinado instante. Corresponde a una especie de reglas estímulo-respuesta. En general, estas funciones son estocásticas.

Una **señal de recompensa** (*reward signal*) es el *reward* que entrega el ambiente al agente en cada paso de tiempo. Ella afecta directamente la manera en que el agente se comportará. Por lo general, el objetivo del agente es *maximizar la recompensa total*, a largo plazo.

La **función de valor** (*value function*) especifica qué es lo bueno a largo plazo, indicando el monto total que puede esperar el agente a largo plazo. Es justamente parte de lo que queremos aprender.

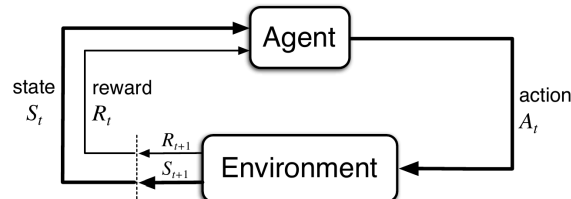
El **modelo del ambiente** es un objeto que imita el comportamiento del ambiente y que permite realizar inferencias de cómo el ambiente se comportará. En el RL podemos encontrar métodos **basados en modelos** (*model-based*), que son justamente aquellos que tienen un modelo del ambiente, y otros métodos más simples, denominados métodos **libres de modelo** (*model-free*).

Procesos de Decisión de Markov Finitos

Este problema define el campo del RL.

Una especificación completa de un ambiente define una **tarea**, esto es, una instancia de RL.

A continuación se empieza a introducir notación. El agente y el ambiente interactúan en instantes de tiempo discretos, $t = 0, 1, 2, \dots$ (aunque existen ideas que consideran el tiempo continuo). En cada instante, el agente recibe una **representación del estado del ambiente**, $S_t \in \mathcal{S}$, donde $\mathcal{S}$ es el **espacio de estados** (*state space*). En esa situación, escoge una **acción** $A_t \in \mathcal{A}(S_t)$, donde $\mathcal{A}(S_t)$ corresponde al conjunto de acciones posibles dado el estado S_t . Como consecuencia, un instante de tiempo después, el ambiente retorna una **recompensa** $R_{t+1} \in \mathcal{R}$, donde $\mathcal{R} \subset \mathbb{R}$ es un conjunto de posibles recompensas. Además, el ambiente le indica el nuevo estado S_{t+1} al agente.



La política del agente la denotamos como $\pi_t(a|s)$, entendida como la probabilidad de que $A_t = a$ dado el estado $S_t = s$. Así, los métodos de RL consistirán en especificar cómo el agente modifica su política a partir de los resultados que obtiene.

Nota 1.2.1. La frontera entre agente y ambiente suele estar más cerca al agente de lo que se cree. Por ejemplo, los motores y sensores de un robot se consideran parte del ambiente (pues pueden entenderse como estados y recompensas), no del agente. Normalmente, basta con preguntarse si el agente *no* puede cambiar algo arbitrariamente. De ser así, entonces ese algo no pertenece al agente. De esta manera, la frontera agente-ambiente corresponde al límite de control del agente, no de su conocimiento. ■

La recompensa es la manera en que comunicamos *qué* es lo que queremos, no *cómo* lo queremos.

En general, buscamos maximizar el **retorno esperado**, esto es, la esperanza de G_t , una función de los retornos. Por ejemplo,

$$G_t = \sum_{j=t+1}^T R_j,$$

donde T es el último paso, si es que tiene sentido que haya un último paso. En dicho caso, cada **episodio** (o *intentos*) termina en un estado especial llamado **estado terminal**. Denotaremos con $\mathcal{S}$ los estados no-terminales y con $\mathcal{S}^+$ todos los estados, incluyendo a los terminales.

Cuando la interacción agente-ambiente no se puede separar en episodios identificables, es decir, no tenemos la noción de «último paso», llamamos a estas tareas *continuing tasks*. En este caso, tendríamos $T = \infty$ para G_t , por lo que podría diverger.

Para que G_t tenga un valor finito, se introduce el factor $0 \leq \gamma \leq 1$ conocido como **tasa de descuento** para definir el **retorno descontado**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Unificación de Notación entre Tareas Episódicas y Continuas

Para tareas episódicas, el estado que se tiene en el instante t y episodio i se denota $S_{t,i}$. Similarmente ocurre con $A_{t,i}$, $R_{t,i}$, $\pi_{t,i}$, T_i . Como muchas veces estamos trabajando dentro de un episodio, se suele omitir el subíndice i .

Además, para unificar la notación del retorno, se pensará que las tareas episódicas tienen infinitos pasos, pero que al llegar al estado terminal se quedan en un *estado absorbente* que transiciona sólo a si mismo con retorno cero.

Procesos de Decisión de Markov

Si el espacio de estados y el de acciones son finitos, entonces estamos ante un **finite MDP**. Un finite MDP se define completamente a partir de: el espacio de estados, el espacio de acciones y por las dinámicas paso a paso del ambiente.

Dado un estado s y una acción a , la probabilidad de llegar al estado s' y obtener una recompensa r se denota por

$$p(s', r|s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}.$$

Al tener todos los valores de p dadas las posibles combinaciones de s', t, s, a , se tienen definidas las *dinámicas del ambiente*.

Dadas estas dinámicas, se puede calcular lo que sea. Por ejemplo, los *retornos esperados de un par estado-acción*,

$$\begin{aligned} r(s, a) &= \mathbb{E}(R_{t+1} \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} r \cdot \Pr(R_{t+1} = r \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a). \end{aligned}$$

Así también se pueden tener las *probabilidades estado-transición* (es decir, la probabilidad de pasar de un estado a otro tomando cierta acción),

$$p(s'|s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = \sum_{r \in \mathcal{R}} p(s', r|s, a).$$

Finalmente, también se pueden obtener los *retornos esperados para un triple estado-acción-siguiente estado*,

$$r(s, a, s') = \mathbb{E}(R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s')$$

$$\begin{aligned}
&= \sum_{r \in \mathcal{R}} r \cdot \Pr(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s') \\
&= \sum_{r \in \mathcal{R}} r \cdot \frac{\Pr(R_{t+1} = r, S_t = s, A_t = a, S_{t+1} = s')}{\Pr(S_t = s, A_t = a, S_{t+1} = s')} \\
&= \sum_{r \in \mathcal{R}} r \cdot \frac{\Pr(R_{t+1} = r, S_{t+1} = s' \mid A_t = a, S_t = s) \cancel{\Pr(A_t = a, S_t = s)}}{\Pr(S_{t+1} = s' \mid A_t = a, S_t = s) \cancel{\Pr(A_t = a, S_t = s)}} \\
&= \sum_{r \in \mathcal{R}} r \cdot \frac{p(s', r \mid s, a)}{p(s' \mid a, s)}.
\end{aligned}$$

Para los MDP finitos podemos construir un **grafo de transición** que permiten resumir las dinámicas visualmente. En ellos existen dos tipos de nodos: los **nodos de estado**, de los cuales se construye uno por cada estado $S \in \mathcal{S}$; y los **nodos de acción**, de los cuales se construye uno por cada par estado-acción. Los nodos de acción tienen entrante una arista dirigida desde un nodo de estado y tienen saliente una (o más) arista(s) dirigida(s) a otro nodo(s) de estado, donde este tipo de aristas presentas anotada la probabilidad y recompensa correspondiente de la transición estado-acción-estado siguiente.

Funciones de Valor

1.3. DreamerV3

1.4. Notas de Composicionalidad

¿Qué no entendí?

1.5. Referencias Semana 0

- **S0 - Notas-ApprComposicional.** (Una pasada para ver qué no conozco, otra para entender todo.)
- Paper DreamCoder: **Mastering Diverse Domains through World Models** (Sólo hasta Sección 2).
- **Li, M. & Vitányi, P. (2008). An Introduction to Kolmogorov Complexity and Its Applications**, 3ª ed. Springer. (C1 y C2)
- **Sipser, M. (2012). Introduction to the Theory of Computation**, 3ª ed. — Capítulos 3 (máquinas de Turing) y 4 (decidibilidad, problema de la parada).
- **Grünwald, P. D. (2007). The Minimum Description Length Principle.** MIT Press (C1)

II Semana 1

2.1. Complejidad de Kolmogorov

Buscamos describir un objeto en una string binaria. Un objeto podría tener muchas descripciones, pero una descripción sólo puede corresponder a un único objeto. Además, podríamos tomar el largo de la descripción más corta y considerarla como la *complejidad* del objeto.

Pero en ese caso podríamos caer en una trampa descrita en la **Richard-Berry paradox**, donde definimos un número natural como «el menor natural que no puede ser descrito en menos de veinte palabras». Si ese número existe, lo acabamos de describir en 13 palabras, contradiciendo la frase que lo define. Si tal número no existe, todos los números pueden describirse en menos de veinte palabras. Este resultado aparece por la *ambigüedad* sistemática de la palabra «descrito» (o, en otros casos, de «definible»). Resolver este tipo de paradojas requiere indicar dónde exactamente nuestro lenguaje «anduvo mal» e indicar restricciones para evitarlas.

Asuma que cada descripción describe a lo sumo un objeto. Así, existe un **método de especificación** $D : Y \rightarrow X$ que asocia, con una descripción $y \in Y$, a lo sumo un objeto $x \in X$. Además, para que las descripciones sean útiles, las asumiremos finitas, así que existe una cantidad numerable de descripciones y , y por tanto, a lo sumo una cantidad numerable de objetos descriptibles.

Para medir la complejidad de un objeto $x \in X$, podríamos escoger aquel que tenga menor costo de transmisión, esto es, el menor largo de y tal que $D(y) = x$. Sin embargo, la complejidad de un objeto dependería directamente de D . Es por esto que, para poder comparar la complejidad de objetos, haremos que la complejidad de x dependa únicamente de él.

Además, para que las especificaciones D sean útiles, será necesario que $D(y)$ sea ejecutable en una **manera efectiva**, es decir, que al menos puede ser ejecutable por humanos o por máquinas.

Asumiremos como resultado que el conjunto de funciones parciales computables contiene una función óptima que minimiza el largo de descripción de cualquier otra función. Denotaremos dicha función con D_0 . Y para cualquier otra función computable D , para cualquier objeto x , existe una descripción de x bajo D_0 que es más corta —hasta una constante aditiva independiente de x — que cualquier otra descripción de x bajo D .

Identificaremos el largo de descripción de x con respecto a una función de especificación fijada D_0 como la **complejidad algorítmica** de x . La optimalidad de D_0 en este sentido quiere decir que la complejidad de un objeto es **invariante** (hasta una constante aditiva independiente de x) bajo la transición de un método de especificación a otro. Este resultado se conoce como

Teorema 2.1.1. (de Invarianza). Existe una máquina de Turing universal (o función parcial computable universal) U tal que, para cualquier otra máquina de Turing (o función computable) F , existe una constante c_F que depende exclusivamente de F y de la elección de U , de modo que para cualquier cadena x ,

$$C_U(x) \leq C_F(x) + c_F,$$

donde

- $C_M(x) = \min\{l(y) : D(y) = x\}$ denota la complejidad de Kolmogorov en la máquina o función $D : Y \rightarrow X$,
- y es la descripción,

- $l(y)$ es la longitud de la descripción. ■

De esta manera, la complejidad de un objeto es un atributo objetivo e inherente a él.

Notación

Si ϕ es una **función parcial** de A a B , entonces para cada $x \in A$, o bien $\phi(x) \in B$, o bien $\phi(x)$ está indefinida. Si $\phi(x) < \infty$, diremos que ϕ **converge**. Por el contrario, si $\phi(x) = \infty$, diremos que ϕ **diverge** o que está **indefinida** en x . Si ϕ converge para cada $x \in A$, entonces diremos que es una **función total**, de otro modo diremos que es una **función estrictamente parcial**. Si cada miembro de B está también en el rango de ϕ , entonces diremos que ϕ **mapea sobre (onto) B** , si no **mapea a (into) B** . Además, si para cada $x \neq y$, ϕ converge y $\phi(x) \neq \phi(y)$, entonces llamamos a ϕ un **mapeo uno-a-uno**, de otro modo la llamamos un **mapeo muchos-a-uno**.

Tendremos los siguientes cuantificadores:

- $\exists$: Existe al menos un elemento.
- $\forall$: Para todo elemento.
- $\exists^\infty$: Existen infinitos elementos.
- $\forall^\infty$: Para todo elemento, excepto para finitos.

$$\neg \exists^\infty x. \phi(x) \iff \forall^\infty x. \neg \phi(x).$$

Nota que podemos hacer el símil entre $\exists^\infty$ con *infinitely often* y entre $\forall^\infty$ con *almost always*.

Strings Binarias

Tendremos nuestro conjunto no vacío $\mathcal{B}$ de **elementos básicos**. Usualmente, $\mathcal{B} = \{0, 1\}$. El conjunto de todas las strings finitas sobre $\mathcal{B}$ es

$$\mathcal{B}^* := \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\},$$

donde ϵ denota la **string vacía**.

La **concatenación** es una operación binaria de elementos en $\mathcal{B}^*$ que toma dos elementos $x, y \in \mathcal{B}^*$ y los lleva a $xy \in \mathcal{B}^*$. Algunas propiedades son:

1. $\mathcal{B}^*$ es cerrada bajo concatenación.
2. La concatenación es asociativa.
3. La concatenación tiene un elemento *unitario* ϵ :

$$\epsilon x = x\epsilon = x.$$

Definiremos una **correspondencia** esencial entre los naturales $\mathbb{N}$ y las strings binarias finitas $\mathcal{B}^*$. Es de notar que no usaremos la representación binaria estándar, pues tiene la desventaja de que, o bien hay strings que no representan números naturales, o bien hay números naturales con múltiples representaciones. Por ejemplo, es bastante conocido que los ceros como prefijos no tienen significado en este sistema: ;010, 10 y 0010 representan al mismo número! Por ello, definimos un mapeo uno-a-uno de $\mathcal{B}^*$ sobre $\mathbb{N}$ asociando cada string con su índice en el **orden lexicográfico por longitud creciente (shortlex)**

$$\begin{aligned}\epsilon &\longleftrightarrow 0 \\ 0 &\longleftrightarrow 1 \\ 1 &\longleftrightarrow 2 \\ 00 &\longleftrightarrow 3\end{aligned}$$

$$\begin{aligned}
01 &\leftrightarrow 4 \\
10 &\leftrightarrow 5 \\
11 &\leftrightarrow 6 \\
&\vdots
\end{aligned}$$

De esta manera, representamos a

$$x = 2^{n+1} - 1 + \sum_{i=0}^n a_i 2^i$$

como $a_n \dots a_1 a_0$. Además, consideraremos al string en $\mathcal{B}^*$ y al número con que se corresponde como exactamente el mismo objeto.

Para una string x , x_i denota el i -ésimo bit y $x_{i:j}$ denota el segmento $x_i x_{i+1} \dots x_j$ de largo $j - i + 1$. El **reverso** lo denotaremos por x^R . El **largo** $l(x)$ de la string finita lo definimos como el número de bits que contiene. En particular, $l(\epsilon) = 0$, $l(x^R) = l(x)$ y $l(xy) = l(x) + l(y)$.

Sea D una función $D : \{0, 1\}^* \rightarrow \mathbb{N}$. Diremos que el dominio de D serán las **palabras código** (*code words*) y su rango el conjunto de **palabras fuente** (*source words*). De esta manera, $D(y) = x$ se interpreta como que y es una code word de la source word x y D es una **función de decodificación**. Al revés, podemos definir una **sustitución de codificación** $E = D^{-1}$ (que no necesariamente una función).

Diremos que una string x es prefijo de z si es que existe una string y tal que $z = xy$. Diremos que $A \subset \{0, 1\}^*$ es **libre de prefijos** (*prefix-free*) si ningún elemento de A es prefijo de otro elemento en A .

Ejemplo 2.1.2. Si $A = \{101, 010, 11, 00\}$ es un conjunto libre de prefijos. ■

Una función $D : \{0, 1\}^* \rightarrow \mathbb{N}$ define un **código prefijo** (*prefix-code*) si su dominio es libre de prefijos.

Ejemplo 2.1.3. Un código prefijo muy utilizado es aquel obtenido al reservar el cero como punto de detenimiento y codificando $x \in \mathbb{N}$ como $1^x 0$. Así, $D : A \rightarrow \mathbb{N}$ tiene

$$A = \{0, 10, 110, 1110, 11110, \dots\}$$

que claramente es libre de prefijos. Además, podemos prefijar un objeto con su largo e iterar la idea para obtener códigos incluso más cortos. Para esto se define

$$E_i(x) = \begin{cases} 1^x 0 & \text{para } i = 0 \\ E_{i-1}(l(x))x & \text{para } i \geq 1. \end{cases}$$

Así, $E_0(x) = 1^x 0$ (que tiene largo $x + 1$), y $E_1(x) = 1^{l(x)} 0x$ (que tiene largo $2l(x) + 1$). [Recuerde que $l(x) \approx \ln x$, así que asintóticamente los E_i tienen largo más corto que los anteriores.] En particular, E_1 tiene una notación especial:

$$\bar{x} := E_1(x) = 1^{l(x)} 0x$$

$$l(\bar{x}) = 2l(x) + 1.$$

Llamamos a $\bar{x}$ la versión **auto-delimitante** (*self-limiting*) de la string x . ■

Computabilidad y Máquinas de Turing

Estudiaremos que todo lo que puede ser razonablemente considerado computable por un *computador humano* (un calculista) usando un procedimiento fijo puede ser computado por una máquina hipotética, una máquina de Turing. Y, como Turing indicó, cualquier proceso que puede ser naturalmente llamado *procedimiento efectivo* es realizado por una máquina de Turing. Esto último se conoce como la **tesis de Turing**. De hecho, todos los intentos de dar

una precisa aunque intuitiva definición de la noción de lo que es un «procedimiento efectivo» (en el sentido más amplio) han resultado ser equivalentes. En este sentido, la **tesis de Church** establece que existe una noción de lo que es la *computabilidad efectiva* independiente de la formalización particular.

Una **máquina de Turing** consiste en un programa finito, llamado **control finito**, capaz de manipular una **cinta**, esto es, una lista lineal de **celdas**, usando un puntero de un acceso, llamado **cabeza**. La cinta tiene dos lados, derecha e izquierda. Además, el control finito tiene un **conjunto de estados** Q finito y cada celda puede contener un 0, un 1 y un **blanco** B . El tiempo se asume discreto, partiendo con la máquina desde el instante cero. En cualquier instante la cabeza está posicionada sobre una celda y la **escanea**. En el instante cero la cabeza está sobre una **celda inicial** y el control finito está en un estado distinguido $q_0 \in Q$ y, además, todas las celdas contienen 0's, excepto por una secuencia finita y contigua de celdas, que se extienden desde la celda inicial, las cuales contienen una secuencia binaria ingresada, llamada **input**.

El aparato puede realizar las siguientes operaciones (sólo una por instante):

1. Escribir un elemento de $A = \{0, 1, B\}$ en la celda que escanea.
2. Mover la cabeza una celda a la derecha o a la izquierda.

Al finalizar cada paso, el control finito toma un estado de Q . Además, el aparato está construido de modo que se comporte de acuerdo a una **lista de reglas** finita, las cuales indican, según el símbolo leído y el estado actual, la operación a realizar *en el siguiente instante* y el estado en que ponerse al terminar la siguiente operación. Cada regla tiene la forma

$$(p, s, a, q),$$

donde:

- p es el estado actual.
- s es el símbolo siendo escaneado.
- a es la acción a ejecutar en el siguiente paso, pudiendo escoger de $S = \{0, 1, B, L, R\}$ (esto es, la máquina o se mueve o escribe, pero no las dos al mismo tiempo).
- q es el estado a tomar al finalizar este instante.

La máquina de Turing es determinista: no pueden haber dos reglas que compartan los mismos dos elementos (es decir, p, s). Además, puede ocurrir que un estado p y un símbolo s no tengan una regla definida. En ese caso, el aparato **para** (*halts*). De esta manera, podemos definir la máquina de Turing mediante un mapeo (o bien, una función parcial) de $Q \times A$ hacia $S \times Q$.

Podemos asociar una función parcial con cada máquina de Turing:

- El input $x = x_1 \dots x_n$ a la máquina se representa mediante una n -tupla $(x_1, \dots, x_n)$, donde cada $x_i \in \{0, 1\}$. (En caso de que $x \in \mathbb{N}$ sea un número, primero se transforma mediante *shortlex*.)
- Se escribe el input en la cinta hacia la derecha partiendo desde la celda inicial.
- El instante en que la máquina para, el output corresponde a la secuencia de símbolos que queda escrita en la cinta. (En caso de que la máquina nunca se detenga, no se produce ninguna salida.)

Así, se ha asociado una máquina de Turing con una función parcial f en el sentido de que $f(x) = y$ siempre que se ingresa x a la máquina y esta tiene como output y y, cuando

la máquina no se detiene, entonces $f(x)$ es indefinido. Por tanto, podemos decir que *toda máquina de Turing computa (o induce) una función parcial*.

Llamaremos a aquella f **función parcial computable**. Si la máquina se detiene para todos los inputs posibles, entonces diremos que f es **total y computable**, o simplemente **computable**.

Definición 2.1.4. (Tesis de Church). La clase de funciones numéricas algorítmicamente computables (en el sentido intuitivo) coincide con la clase de funciones parcialmente computables. ■

La tesis de Church se piensa intuitivamente como que «dada una descripción de un procedimiento en términos de un conjunto informal de instrucciones, nosotros podemos derivar un procedimiento formal en términos de máquinas de Turing».

Definición 2.1.5. (Máquina de Turing Universal). Una máquina de Turing U es **universal** si puede imitar el comportamiento de cualquier otra máquina de Turing T . ■

Está demostrado que dichas máquinas existen (de hecho, son infinitas) y que pueden ser efectivamente construidas. Bastaría con una descripción del programa finito de T y el input a ingresar a ella, para ingresarlos en la cinta inicial de U . Para ejecutar las acciones que haría T , U iría leyendo el programa de T y las ejecutaría en su propia cinta.

Existe un emparejamiento (efectivo computable) uno-a-uno entre los naturales y las máquinas de Turing llamado **enumeración efectiva**, $T_1, T_2, \dots$. De hecho, esta enumeración (efectiva) permite crear una enumeración efectiva de funciones parciales computables (¿Por qué?) $\phi_1, \phi_2, \dots$.

Existe una diferencia entre el *nombre* para ψ y una *función* ψ . Un nombre para ψ puede ser un algoritmo que la calcule con una máquina de Turing, o puede ser un **índice** $i \in \mathbb{N}$ tal que ψ es igual a ϕ_i es la enumeración efectiva. [Cada función parcial computable ψ ocurre muchas veces dada una enumeración efectiva, pues podemos construir distintas máquinas de Turing que para el mismo input produzcan el mismo output.]

Definición 2.1.6. La función parcial computable $\nu^{(2)}(i, x)$ computada por la máquina universal U se llama la **función parcial universal computable**. ■

Un conjunto $A \subseteq \mathbb{N}$ es **computablemente enumerable** (c.e.) si es vacío o es el rango de alguna función f total computable. En ese caso, diremos que f **enumera** a A . Diremos además que un conjunto $B \subseteq \mathbb{N}$ es **co-computablemente enumerable** (co-c.e.) si su complemento $A = \overline{B}$ es c.e.. Un conjunto $C \subseteq \mathbb{N}$ es **computable** ssi su función característica es total computable, esto es, $f = \mathbb{1}_A$ es total computable. Así que todos los conjuntos computables son c.e. y co-c.e..

Teoría de la Información

2.2. MDL

2.3. Cálculo Lambda

2.4. DreamCoder

Domain-Specific Languages (DSL)

Síntesis Inductiva de Programas

$\{\mathcal{Q}_{x_i}\}$

Typed Grammars

Definición 2.4.1. (Expression tree).

■

Connection to Helmholtz Machine and VAEs

Helmholtz Machine

Variational Inference Primer

Diferencias y similitudes

Resultados Experimentales

2.5. Flat Minima

Resultados Experimentales

2.6. Glosario de la Semana

Weight Decay

Hinton & van Camp

Optimal Brain Surgeon / Damage (OBS / OBD)

Algoritmo de Gibbs

Distancia de Kullback-Leibler

III Semana 2

Los dos papers de esta semana intentan obtener lo mejor de dos mundos de síntesis de programas, el enfoque simbólico (o inductivo) y el enfoque neuronal (o transductivo).

3.1. Searching Latent Program Spaces

El paper presenta la arquitectura **Latent Program Network (LPN)** que busca sintetizar programas a partir de pocos ejemplos y ajustarlos en caso de que la primera intuición no alcance. Para ello, toma elementos de dos acercamientos a la resolución de ARC-AGI (que los autores defienden como una buena manera de medir la inteligencia por medir la capacidad de adaptarse a nuevas tareas):

- **Acercamiento inductivo** / *Síntesis de programas simbólica*: Se produce un programa explícito, pero el espacio de búsqueda es demasiado grande, por lo que requiere un lenguaje de dominio específico (DSL) diseñado a mano que limite las opciones, algo poco escalable a tareas del mundo real. LPN de aquí usa la capacidad de búsqueda durante el momento de prueba.
- **In-context/transductivo** / *Aprendizaje profundo*: Las redes neuronales son muy escalables, pero luego de entrenarse fijan sus pesos, por lo que suele ser necesario reentrenar y, con pocos datos, suelen overfittear. Además, tienden a ser inconsistentes en reproducir los ejemplos que se les dieron. LPN de aquí toma la predicción principal.

LPN representará los programas implícitos en un espacio continuo latente de programas y usará backpropagation en su fase de *fine-tuning* con una fracción del total de parámetros.

El paper propone realizar test-time training, que por lo general es muy costoso pues implica la actualización de millones de parámetros, en un *espacio latente continuo y compacto* que la propia red aprendió (en vez de hacerlo en el espacio de parámetros de la red neuronal o en el espacio de programas de un DSL).

Background

Se define el **espacio de programas** Y formulados a partir de un DSL. Cada tarea está definida por un **conjunto de especificación** X , donde cada **especificación** $X_m \in X$ se describe como un conjunto de pares input-output,

$$X_m = \{(x_1^m, y_1^m), \dots, (x_n^m, y_n^m)\}.$$

Un programa $f \in Y$ resuelve la tarea asociada a X_m si

$$\forall j \in [n], \quad f(x_j^m) = y_j^m.$$

Denotaremos con F_m a la función verdadera que generó X_m , es decir, $F_m(x_j^m) = y_j^m$ para cada $j \in [n]$.

El problema de generalización se define a partir del conjunto

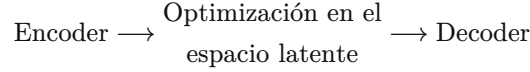
$$P_m = \{(x_1^m, y_1^m), \dots, (x_n^m, y_n^m), x_{n+1}^m\},$$

donde se buscará predecir el output correspondiente de x_{n+1}^m .

El paper menciona que, de no limitar la complejidad de Kolmogorov del programa, podemos potencialmente encontrar un programa para cualquier especificación, aun si corresponde o no al programa original F_m . (¡Esto es una gran puerta al overfitting!)

Arquitectura LPN

El proceso de inferencia de LPN ocurre en tres etapas:



El encoder generará una primera «intuición», la optimización en el espacio latente buscará refinar dicha intuición y el decoder ejecutará el programa refinado sobre el nuevo ejemplo.

El Espacio Latente de Programas

Un **espacio latente** es un espacio vectorial de dimensión reducida donde una red neuronal representa de forma comprimida y abstracta la información relevante de una entrada mucho más compleja.

Autoencoder Variacionales (VAE) y LPN

Un **autoencoder variacional (VAE)** es una red neuronal generativa que toma datos de entrada, los transforma a un espacio latente continuo según una distribución de probabilidad (lo que permitirá generar nuevos datos) y toma los puntos de dicho espacio para reconstruirlos o transformarlos en nuevos datos.

Más formalmente, un VAE tiene:

- Un **encoder** $q_\phi(z \mid x, y)$: dado un dato (x, y) de entrada, produce parámetros de media μ y varianza Σ de una distribución de probabilidad Normal multivariada sobre los posibles valores de un vector z en el espacio latente de programas. Además, el encoder procesa cada par por separado y luego los promedia, de modo que el LPN sea invariante a la *permutación de ejemplos*. ϕ corresponde a los pesos de la red neuronal del encoder.
- Un **decoder** $p_\theta(y \mid x, z)$: dado un programa latente z y un input nuevo x , reconstruye los datos de salida y . Se construye maximizando la verosimilitud de reconstruir bien los datos. θ corresponde a los pesos de la red neuronal de decoder.

El VAE permitirá modelar la ambigüedad de los muchos programas posibles con la distribución de probabilidad.

La Optimización en el Espacio Latente

Tendremos

$$\underbrace{z \sim q_\phi(z \mid x, y)}_{\text{Encoder}} \quad \underbrace{z' = f(p_\theta, z, x, y)}_{\text{Optimización latente}} \quad \underbrace{\hat{y} \sim p_\theta(y \mid x, z)}_{\text{Decoder}}$$

El paso intermedio parte desde z , donde se busca un mejor programa z' que explique, bajo el actual decoder, de mejor manera los datos observados.

Dados datos $\{(x_i, y_i)\}_{i=1}^n$, el proceso de búsqueda $z' = f(p_\theta, z, x, y)$ consiste en encontrar un z' que satisfaga

$$z' \in \arg \max_z \sum_{i=1}^n \log p_\theta(y_i \mid x_i, z),$$

esto es, se busca el programa que más probabilidades tenga de producir resultados correctos bajo el decoder actual y los datos que se tienen.

Se proponen dos maneras de encontrar z' :

- Por **muestreo**: se generan K programas z_k muestreados (de alguna distribución, ya sea $p(z)$ o la posteriori $q_\phi(z \mid x_i, y_i)$) y se elige el que tenga mayor log-verosimilitud. Su eficiencia decrece exponencialmente respecto a la dimensión del espacio latente.

- Por **Ascenso de Gradiente**: el decoder es una red neuronal diferenciable y, en particular, también su log-verosimilitud respecto a z . Se inicializa en z'_0 (promedio de todos los pares latentes sampleados del encoder) y se itera con

$$\forall k \in [1, K], \quad z'_k = z'_{k-1} + \alpha \cdot \nabla_z \sum_{i=1}^n \log p_\theta(y_i | x_i, z)|_{z=z'_{k-1}}.$$

El ascenso de gradiente aquí se realiza en el espacio latente de programas. Como el espacio es de menor dimensión que el de los parámetros de la red, la optimización es mucho más barata.

Entrenamiento del Encoder y del Decoder

Aquí se trata cómo se obtienen los valores de ϕ y de θ .

Primero, para entrenar al encoder se utiliza la estrategia «**leave-one-out**» (LOO), donde, para reconstruir cada y_i :

1. Con cada $j \neq i$, se muestrea $z_j \sim q_\phi(z | x_j, y_j)$ con el encoder.
2. Se agregan promediándolos: $\bar{z} = \frac{1}{n-1} \sum_{j \neq i} z_j$.
3. Se obtiene un z'_i con optimización en el espacio latente (e.g. con ascenso de gradiente).
4. Se calcula la log-verosimilitud de predecir correctamente y_i dado x_i y z'_i con el decoder (fijo en este instante).

La función objetivo a entrenar

$$\mathcal{L}_{\text{total}}(\phi, \theta) = \mathcal{L}_{\text{rec}}(\phi, \theta) + \beta \mathcal{L}_{\text{KL}}(\phi)$$

se subdivide en:

- **Pérdida de reconstrucción** $\mathcal{L}_{\text{rec}}$:

$$\mathcal{L}_{\text{rec}}(\phi, \theta) = \sum_{i=1}^n -\log p_\theta(y_i | x_i, z'_i)$$

Mide qué tan mal predice el *decoder* las salidas correctas.

- **Pérdida de regularización** $\mathcal{L}_{\text{KL}}(\phi)$:

$$\mathcal{L}_{\text{KL}}(\phi) = \sum_{i=1}^n D_{\text{KL}}(q_\phi(z | x_i, y_i) \parallel \text{Normal}(0, I))$$

Mide qué tan lejos está la distribución que produjo el *encoder* de una distribución Normal (gracias a la **divergencia de Kullback-Leibler**), ayudando a que esta sea lo suficientemente suave para poder ser diferenciada.

- Factor β : Regula qué tanta importancia tiene un término por sobre otro.

Se menciona un **truco de reparametrización**, donde en vez de muestrear un $z \sim \text{Normal}(\mu, \Sigma)$, se muestrea ruido $\epsilon \sim \text{Normal}(0, I)$ y luego se transforma linealmente

$$z = \mu + \epsilon \cdot \Sigma,$$

de modo que dicho z sea diferenciable¹.

Experimentos y Resultados

PENDIENTE

¹¿Porque vamos a diferenciar $\mathcal{L}_{\text{total}}$ respecto a ϕ ?

Limitaciones

PENDIENTE

Se sabe que tienen problemas para generalización composicional.

Preguntas

- ¿Cómo evita el sobreajuste?
- No entiendo del todo el flujo al entrenar. ¿Cuándo se usa LOO y cuándo ascenso de gradiente? ¿Se entrenan el encoder y el decoder una sola vez al principio y ya, o se vuelven a entrenar luego de hacer ascenso de gradiente con un ejemplo?
- ¿El promedio en LOO es el que introduce el término de invariante ante permutaciones?
- ¿Qué es el *early stopping*? ¿Por qué ayuda a no sobreajustar?
- ¿Cómo se ahorra costos el modelo?

Glosario

- **Fine-tuning**: Proceso de adaptar los parámetros de una red una vez esta ya ha sido entrenada.
- **Test-time training**: Capacidad de los modelos de aprender (optimizar o hacer búsqueda temporal) luego de ya haber sido «entrenados» (i.e. durante la fase de inferencia), pudiendo ajustarse a los ejemplos de una tarea nueva para resolverla momentáneamente.
- **Few-shot learning**: aprendizaje con pocos casos.

3.2. Gradient-Based Program Synthesis with Neurally Interpreted Languages

Introducen el concepto de **Latent Adaptation Networks (LANs)**, que son redes con encoder y decoder que aprenden un espacio latente para representar el espacio de modelos (o programas). Una característica esencial es que en test-time se busca en el espacio latente para adaptarse y mejorar los resultados.

Los autores presentan la arquitectura **Neural Language Interpreter (NLI)** que justamente es del tipo LAN, pero a diferencia del paper anterior, este aprovechará tokens para construir el programa en el espacio latente (y no representarlo como un único vector), de modo que se tenga mayor poder de generalización. Además, esta característica permite que el modelo pueda resolver problemas más difíciles (que requieren más tokens) que los que se le presentaron durante el entrenamiento.

Formalización

Se formaliza la tarea como un **program induction**. Dado un programa p que genera los pares $S = \{(x_i, y_i)\}_{i=1}^n$ (es decir, $p(x_i) = y_i$), buscamos un modelo M que prediga con $M(S, x_{n+1})$, para un nuevo input x_{n+1} , el valor de $p(x_{n+1})$. Se denotará con $\mathcal{P}$ el espacio de posibles programas y se definen distribuciones P_{train} y P_{test} para entrenar y testear, respectivamente, la arquitectura, de modo que se pueda verificar su generalización.

Inferencia Discreta Secuencial

NLI aprende un lenguaje de programación discreto y «simbólico» (*symbolic-like*) para representar programas como secuencias de tokens (de longitud variable) que son ejecutadas por un ejecutor neuronal diferenciable. (¡Importa mucho que sea diferenciable para la optimización que se realizará!)

Objetivo de Entrenamiento

Para aprender el espacio latente con el que representar problemas, se usa una arquitectura encoder-decoder. El encoder, **inductor de programa** q_ϕ , infiere una representación latente de programas desde unos pares input-outputs, la cual el decoder, **interpretador neuronal** p_θ , ejecuta para predecir outputs dado un input.

Nuevamente se usa la estrategia de *leave-one-out* sobre *mini-batches* de tamaño B , donde cada especificación contiene m pares input-output. Así, q_ϕ induce un espacio latente usando $m - 1$ pares y el error de predicción se calcula con aquel par dejado fuera. Definimos, para la b -ésima especificación y el i -ésimo par, $\mathcal{S}_{b,i} := \mathcal{S}_b \setminus \{(x_{b,i}, y_{b,i})\}$. Para cada $\mathcal{S}_{b,i}$ buscamos maximizar la verosimilitud de predecir $y_{b,i}$ a partir de $x_{b,i}$, mientras se regulariza para reutilizar tokens. Así, la función de pérdida a minimizar es

$$\mathcal{L}(\phi, \theta; \mathcal{S}) = \frac{1}{B} \sum_{b=1}^B \left(\frac{1}{m} \sum_{i=1}^m \mathcal{L}_{\text{recon}}(\phi, \theta; x_{b,i}, y_{b,i}, \mathcal{S}_{b,i}) + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}(\phi; \mathcal{S}_b) \right)$$

donde

- $\mathcal{L}_{\text{recon}}$ es la **pérdida de reconstrucción**: asegura que el programa sea lo suficientemente expresivo.

$$\mathcal{L}_{\text{recon}}(\phi, \theta; x, y, S) := -\log p_\theta(y \mid x, q_\phi(S)).$$

- $\mathcal{L}_{\text{reg}}$ es la **pérdida de regularización del encoder**: motiva la reutilización de tokens ya descubiertos en el vocabulario de tamaño K (¡esto motiva la composicionalidad!).

$$\mathcal{L}_{\text{reg}}(\phi; \mathcal{S}) := \sum_{k=1}^K \sum_{j=1}^N \left[1 - \exp \left(\sum_{b=1}^B \sum_{i=1}^m \log \left(1 - q_\phi^{(k,j)}(\mathcal{S}_{b,i}) \right) \right) \right]$$

- N : número de posiciones de tokens en el programa.
- $q_\phi^{(k,j)}(\mathcal{S}_{b,i})$: probabilidad asignada al token k en la posición j por el encoder.

El $1 - \exp$ consiste en la probabilidad de que el token sea usado al menos una vez en el batch.

Preguntas

- ¿**Test-time** y **tiempo de inferencia** son lo mismo? En este contexto sí. Van contrarios al **training-time**.
- ¿Qué es una **ablación**? Es un experimento del modelo, donde se remueve un componente de él para entender qué tan necesario es.
- ¿Aquí no se usa un muestreo desde una Normal con media μ y varianza Σ ?
- ¿ $\mathcal{S}_B \subset S$?
- ¿Qué es un **batch**?

Glosario

- **Transformer:** Arquitectura de red neuronal diseñada específicamente para procesar y generar datos secuenciales, como el lenguaje natural. Consisten en un *encoder* (que convierte la entrada en una matriz) y en un *decoder* (que genera una secuencia final en base a lo que generó el decoder y escogiéndola con mayor probabilidad de aparición).

3.3. Diferencias entre ambas investigaciones